

Aula 01 - Fundamentos de APIs e Leitura de Documentação Técnica

Descrição: Nesta aula de abertura, os estudantes exploram a arquitetura Cliente-Servidor e o conceito de API por meio da analogia didática do garçom e do restaurante. Na sequência, aprendem a interpretar documentações técnicas padronizadas (Swagger/OpenAPI e Redoc), identificando rotas, métodos HTTP, parâmetros e respostas em formato JSON. A aula é finalizada com uma prática guiada de testes reais no navegador utilizando a BrasilAPI e a atividade investigativa "Exploradores de APIs".

Nesta primeira etapa da nossa **Aula 01**, vamos entender o propósito desta Unidade Curricular, como ela se conecta com a formação de vocês e quais conhecimentos seremos capazes de dominar ao longo das nossas aulas.

1. Visão Geral da Unidade Curricular

Nome da UC: Interação com APIs.

Carga Horária Total: 40 horas (11 aulas).

Objetivo Geral: Desenvolver capacidades técnicas e socioemocionais para que vocês consigam construir aplicações Front-End capazes de consumir serviços do servidor e exibir dados de forma dinâmica e segura para o usuário.

Função Profissional: Esta UC integra o perfil profissional para produzir interfaces web com padrões de qualidade, usabilidade, interatividade, robustez e segurança da informação.

2. Conhecimentos da Unidade Curricular

Ao longo das nossas 11 aulas, exploraremos um conjunto de conteúdos fundamentais para a integração de sistemas web. Abaixo está o resumo de todos os conhecimentos que faremos parte do nosso aprendizado:

- **Serviços Server-Side e Documentação de APIs:**
 - Estudo e leitura de documentações técnicas de APIs públicas (modelos, componentes e mensagens de retorno).
 - Utilização de ferramentas e padrões de mercado, como Swagger/OpenAPI e Postman Docs.
- **Protocolo HTTP e Ciclo de Operações (CRUD):**
 - Entendimento da estrutura de mensagens HTTP (headers, body JSON e códigos de status 2xx a 5xx).
 - Operação de Consulta (GET) com requisições assíncronas em JavaScript (fetch, Promises e async/await).
 - Operação de Criação (POST) com envio de dados serializados em JSON.
 - Operações de Atualização e Exclusão de registros (PUT, PATCH e DELETE) sincronizados na interface.
- **Tratamento de Falhas e Resiliência:**
 - Captura de exceções e tratamento defensivo de erros (try/catch) no lado do cliente.
 - Feedback visual e amigável ao usuário em casos de instabilidade ou falhas de rede.
- **Segurança da Informação e Autenticação:**
 - Compreensão dos três pilares da segurança: Integridade, Disponibilidade e Confidencialidade.
 - Práticas de controle de acesso, criptografia, auditoria e backups.
 - Implementação de fluxo de login com autenticação JWT (JSON Web Token) e envio no cabeçalho Authorization: Bearer.
- **Desenvolvimento Prático e Projeto Integrador:**

- Mapeamento de endpoints e desenvolvimento de um painel funcional (Situação de Aprendizagem).
- Planejamento, codificação, testes e apresentação de um projeto completo integrando Front-End e APIs.
- **Autogestão e Atitudes Socioemocionais:**
 - Atitude proativa, disciplina e organização para solução de problemas complexos de software.
 - Trabalho em equipe, escuta ativa e fundamentação de decisões técnicas baseadas em evidências.

Pronto para o próximo passo?

Esta foi a nossa introdução geral à Unidade Curricular. Na sequência da aula, iniciaremos o nosso conteúdo técnico explorando a **Arquitetura Cliente-Servidor** e a **Leitura de Documentações de APIs**.

Tópico 1: Conceitos Fundamentais de Comunicação Web

1. O que é uma API? (*Application Programming Interface*)

1.1. Definição e Conceito Geral

- **Significado da Sigla:** *Application Programming Interface* (Interface de Programação de Aplicação).
- **Definição Geral:** É um conjunto de regras, padrões, especificações e protocolos de programação que permite a comunicação e a troca de dados entre dois ou mais sistemas de software distintos, sem que um precise conhecer a implementação interna do outro.
- **Perspectiva de Sistemas Distribuídos:** Em arquiteturas de rede, as APIs atuam como conectores que mediam a comunicação e a transferência de representações de dados entre diferentes componentes de software.

1.2. Analogia Didática: O Garçom e o Restaurante

-  **O Cliente (Usuário / Aplicação):** Representa o cliente no restaurante consultando o cardápio e fazendo o pedido.
-  **A API (O Garçom):** Atua como o intermediário que recebe o pedido do cliente, leva a solicitação até a cozinha e traz o prato pronto de volta.
-  **O Servidor (A Cozinha):** É o sistema centralizado que recebe a ordem, processa a solicitação e consulta a base de dados para preparar a resposta.

[ Cliente / Front-End] —(Faz o Pedido)—> [ API / Garçom] —> [ Servidor / Cozinha]

[ Cliente / Front-End] <—(Entrega a Resposta)— [ API / Garçom] <— [ Servidor / Cozinha]

1.3. O Papel da API no Modelo Cliente-Servidor

- **Intermediária Desacoplada:** A API estabelece uma ponte padronizada de comunicação via protocolo HTTP/HTTPS entre a interface *Front-End* e os serviços do *Back-End*.
- **Contrato de Serviço:** A API estabelece formalmente quais solicitações (*requests*) o servidor aceita.
- **Padronização de Envio:** Define como os dados devem ser enviados, especificando parâmetros, cabeçalhos e corpo (*payload*) em formatos como JSON ou XML.
- **Estruturação de Resposta:** Garante a previsibilidade da estrutura das mensagens de resposta (*responses*) do servidor.

1.4. Principais Objetivos e Benefícios

- **Reutilização e Eficiência:** Permite reaproveitar dados e regras de negócio sem recriar funcionalidades do zero, tais como integrações com gateways de pagamento ou mapas.
- **Segurança e Abstração:** Funciona como uma camada de proteção, expondo apenas os dados estritamente necessários e mantendo a base de dados e as regras internas protegidas.
- **Manutenibilidade:** Mudanças na estrutura interna do servidor não quebram o código do cliente, desde que a interface gráfica respeite o contrato estabelecido pela API.

1.5. Tipos de Acesso às APIs

- **Públicas (Abertas):** Acessíveis a qualquer desenvolvedor ou aplicação externa, como APIs de previsão do tempo, consulta de CEP ou moedas.
- **Privadas (Internas):** Utilizadas exclusivamente dentro da infraestrutura de uma organização para integrar microssistemas ou sistemas legados.
- **Parceiras (Partner APIs):** Expostas com controle de acesso restrito e autenticado para empresas parceiras homologadas.

2. Fundamentos da Arquitetura Cliente-Servidor (Client/Server)

2.1. O Princípio da Separação de Responsabilidades

- **Definição da Arquitetura:** É um modelo de computação distribuída que divide o processamento entre dois papéis principais: quem solicita a informação/serviço e quem processa e responde com a informação.
- **Separação de Responsabilidades (Separation of Concerns):** A arquitetura isola as preocupações associadas à interface gráfica das preocupações de armazenamento e processamento de dados.
- **Evolução Independente:** Esta separação permite que a camada de apresentação seja alterada sem impactar a persistência dos dados, ou que a infraestrutura de dados seja modificada sem quebrar a experiência do usuário.
- **Gestão de Complexidade:** Reduz a complexidade do sistema ao estabelecer limites operacionais claros entre os componentes conectados na rede.

2.2. Os Dois Papéis Principais

Papel	Conceito	Responsabilidades do Sistema	Exemplos Práticos
 Cliente (Client-side)	A interface com a qual o usuário interage diretamente.	• Capturar interações do usuário. • Disparar solicitações de dados. • Renderizar e exibir informações na tela.	Navegadores Web (Chrome, Firefox), aplicativos móveis e programas desktop. Melhora a portabilidade da interface entre plataformas.
 Servidor (Server-side)	O sistema centralizado que armazena recursos e regras.	• Processar regras de negócio. • Consultar e manipular o Banco de Dados. • Validar permissões e autenticação.	Servidores Node.js, Python, Java, Apache e Bancos de Dados Relacionais. Simplifica o processamento na ponta.

2.3. O Ciclo da Comunicação: Request e Response

1. Requisição (Request) ==>

[ Cliente]

[ Servidor]

<=== 2. Resposta (Response)

1. **Requisição (Request):** O cliente dispara uma mensagem formatada pedindo dados ou uma operação específica (ex.: "Servidor, envie a lista de entregas da minha conta").
2. **Processamento:** O servidor recebe a mensagem, autentica a requisição, aplica as regras de negócio e executa a consulta no banco de dados.
3. **Resposta (Response):** O servidor envia uma resposta com os dados no formato combinado ou relata um erro de processamento (ex.: "Aqui estão os dados" ou "Acesso negado").

2.4. Vantagens Estruturais da Arquitetura

- **Segurança e Centralização:** Os dados sensíveis e as regras de negócio permanecem protegidos no servidor, inacessíveis diretamente do navegador.
- **Escalabilidade:** O servidor e a interface gráfica podem evoluir e expandir de forma isolada, otimizando o uso dos recursos computacionais.
- **Reutilização:** Uma mesma API centralizada pode atender simultaneamente a um site web, um app mobile e um painel de administração.

3. Momento de Interatividade: Teste de Entendimento

Cenário do Exercício: Imagine que um usuário esteja em um e-commerce de logística e clique no botão "**Ver meus pedidos**".

Pergunta para a Turma: O que acontece na arquitetura Cliente-Servidor nesse exato instante? Qual é a função do **Cliente** e qual é a função do **Servidor**?

- **Atuação do Cliente (Front-End):** Captura o evento do clique, constrói a requisição HTTP pedindo a lista de pedidos do usuário e envia via API. Ao receber a resposta, renderiza a lista na tela.
- **Atuação do Servidor (Back-End):** Recebe o pedido, confirma se o usuário tem permissão para visualizar aquela conta, busca os registros de compras no Banco de Dados e retorna uma lista formatada em JSON.

Tópico 2: Exploração e Leitura de Documentação Técnica

1. Por que a Documentação é a "Bússola" do Desenvolvedor?

1.1. A Analogia do Manual de Instruções

- Imagine tentar montar um móvel complexo ou um brinquedo repleto de engrenagens sem nenhum manual: o resultado provável é frustração, peças encaixadas no lugar errado e perda de tempo.
- Tentar integrar ou consumir uma API sem consultar a sua documentação técnica é exatamente o mesmo que programar "no escuro".
- A documentação técnica é o **contrato formal** entre o desenvolvedor e o servidor, especificando com

clareza o que pedir, como pedir e o que esperar como resposta.

1.2. O Custo de Programar "No Escuro"

- **Erros de Requisição:** Falta de clareza gera estruturas malformadas de dados enviadas ao servidor.
- **Perda de Produtividade:** Testes de tentativa e erro consomem tempo útil de desenvolvimento.
- **Tratamento Falho de Exceções:** Sem conhecer os códigos de erro do servidor, a interface gráfica não consegue exibir mensagens amigáveis ao usuário.

2. Padrões e Ferramentas de Documentação do Mercado

No mercado de desenvolvimento de software, duas abordagens principais se destacam para documentar interfaces RESTful:

Ferramenta / Padrão	O que é?	Ponto Forte Didático / Operacional
 Swagger / OpenAPI Specification (OAS)	Um padrão universal e aberto para descrever e estruturar APIs REST. O Swagger UI gera uma interface visual interativa em cima desse padrão.	Permite testar requisições diretamente na própria página da documentação por meio do botão <i>Try it out</i> .
 Postman Docs	Documentação gerada automaticamente a partir de coleções de requisições (<i>collections</i>) criadas no software Postman.	Apresenta exemplos de código prontos e funcionais em diversas linguagens de programação (JavaScript, Python, cURL, etc.).

3. Anatomia Visual: Os 4 Componentes Indispensáveis

Ao abrir qualquer documentação técnica de API, o desenvolvedor deve buscar imediatamente quatro componentes vitais de mapeamento:

 1. Endpoints e Métodos HTTP (Rotas e Ações)
 2. Parâmetros da Requisição (Path, Query, Headers)
 3. Modelo de Dados e Payload Body (Estrutura JSON)
 4. Cenários de Respostas (Status Codes & Responses)

1. Endpoints e Métodos HTTP

- **Endpoint (Rota):** O endereço/caminho da URL onde o recurso está localizado no servidor (ex.: `/api/cep/v1`).
- **Método HTTP (Verbo):** A ação semântica que indica a intenção do cliente (ex.: `GET` para consultar ou `POST` para criar).

2. Parâmetros da Requisição (*Parameters*)

- **Path Parameters:** Identificadores obrigatórios inseridos diretamente na estrutura da URL (ex.:

/api/cep/v1/{cep}).

- **Query Parameters:** Filtros ou modificadores opcionais anexados ao final da URL após o símbolo `?` (ex.: `/encomendas?status=Pendente`).
- **Headers (Cabeçalhos):** Metadados de contexto da requisição, como formato de dados ou chaves de autenticação (ex.: `Content-Type: application/json`).

3. 📦 Modelo de Dados e Payload Body

- Refere-se à estrutura e tipagem do objeto (geralmente em formato JSON) transmitido no corpo (*body*) da requisição, especialmente em operações de criação (`POST`) e atualização (`PUT/PATCH`).

4. 💬 Cenários de Respostas (*Responses & Status Codes*)

- O mapa do que o servidor retorna tanto em casos de sucesso quanto em casos de falha:
 - **200 OK / 201 Created:** A solicitação foi processada com sucesso.
 - **400 Bad Request:** Dados enviados pelo cliente estão incorretos ou malformatados.
 - **404 Not Found:** O recurso ou endereço solicitado não existe no servidor.

Passo a Passo Guiado: Explorando a BrasilAPI 🇧🇷

Este guia prático foi feito para orientar você, passo a passo, no uso do navegador web para explorar e testar a documentação de uma API de forma interativa.

Passo 1: Abrir o Navegador e acessar a Documentação Técnica da BrasilAPI

- **O que fazer:** Clique na barra de endereços do seu navegador, digite o link `[https://brasilapi.com.br/docs](https://brasilapi.com.br/docs)` e aperte a tecla **Enter**.
- **O que você verá na tela:** A página oficial da **BrasilAPI**, estruturada no padrão visual **Swagger UI / OpenAPI**. Você verá um índice com vários serviços disponíveis (CEP, CNPJ, DDD, Feriados, etc.).

Passo 2: Localizar o Serviço de Busca de CEP

- **O que fazer:** Role a página para baixo até encontrar o bloco rotulado como `cep-v1` ou `CEP`.
- **Identificando os componentes na tela:**
 - **O Verbo HTTP:** Observe a caixa colorida em azul com o texto `GET`. O verbo `GET` indica que este serviço serve apenas para **consultar** dados.
 - **O Endpoint (Rota):** Ao lado do botão azul, você verá o caminho `/api/cep/v1/{cep}`. Isso mostra o endereço do servidor onde a informação do CEP está guardada.

Passo 3: Como Testar no Navegador

- **Teste direto na Barra de Endereços do Navegador**
 1. Abra uma nova aba no seu navegador.
 2. Digite a URL do serviço juntamente com a rota e o CEP de teste:
`(https://brasilapi.com.br/api/cep/v1/89201000)`.
 3. Aperte **Enter**. O navegador fará a requisição `GET` real ao servidor e exibirá a resposta em JSON na tela.
 4. Para testar o erro **404**, mude os números no final da URL para um CEP inexistente:
`(https://brasilapi.com.br/api/cep/v1/99999999)`

Passo 4: Analisar a Resposta de Sucesso (Response Body JSON)

- **O que fazer:** Role a página um pouco para baixo até a seção **Responses** (Respostas).
- **Interpretando os resultados:**
 - **Server Response Code:** Você verá o código **200**. O código **200 OK** significa que o pedido foi processado com sucesso.
 - **Response Body (Corpo da Resposta):** Você verá um bloco de texto formatado como um objeto **JSON**:

```
JSON
{
  "cep": "89201000",
  "state": "SC",
  "city": "Joinville",
  "neighborhood": "Centro",
  "street": "Rua do Príncipe"
}
```

- **Entendendo a estrutura do JSON:**
 - Cada linha contém uma **chave** entre aspas à esquerda (ex: **"city"**) e o seu respectivo **valor** à direita (ex: **"Joinville"**).
 - É exatamente essa estrutura textual que o nosso programa em JavaScript vai ler e transformar nos textos que o usuário enxerga na tela do site!

Passo 5: Simular e Testar Cenários de Erro

Para entender como a API se comporta quando algo dá errado, faremos dois novos testes:

- **Teste A:** CEP inexistente
 1. No mesmo campo do parâmetro **{cep}**, apague o número e digite **99999999** (formato numérico válido de 8 dígitos, mas que não existe no Brasil).
 2. Clique em **Execute**.
 3. **Resultado:** O código retornado será **404 Not Found**, e a mensagem no JSON explicará que o CEP não foi encontrado na base de dados.
- **Teste B:** Formato de CEP Inválido
 1. Digite no campo do parâmetro a palavra **abc** ou um número incompleto como **123**.
 2. Clique em **Execute**.
 3. **Resultado:** O código retornado será **400 Bad Request**, indicando que a requisição enviada pelo cliente está incorreta ou fora do formato esperado.

Resumo dos Códigos Aprendidos no Teste

Código HTTP	Significado	O que indica no teste?
200 OK	Sucesso	CEP encontrado e dados retornados em JSON.
400 Bad Request	Erro do Cliente	CEP enviado em formato incorreto (letras ou dígitos incompletos).

404 Not Found

Não Encontrado

CEP possui formato numérico válido, mas não existe no banco de dados.

Atividade Interativa: "Exploradores de APIs"

Agora é a hora de exercitar a leitura técnica da documentação com três desafios curtos em sala de aula:

Desafio 1: Busca de CNPJ

- **Missão:** Acesse a documentação da BrasilAPI e identifique qual é o método HTTP e qual é a rota exata (*endpoint*) utilizada para consultar os dados cadastrais de um **CNPJ**.

Desafio 2: Leitura de Status de DDD

- **Missão:** Localize o serviço de busca por **DDD** na documentação e identifique qual é o código de status HTTP retornado quando o DDD digitado não existe no Brasil.

Desafio 3: Mapeamento de Payload de Feriados

- **Missão:** Encontre o *endpoint* de **Feriados Nacionais** e liste quais campos de informação o servidor devolve para cada feriado retornado na lista JSON.